

Chapitre 1 : Introduction aux Systèmes d'Exploitation et Entrées/Sorties

I Introduction aux Systèmes d'Exploitation

A Rôle du système d'exploitation

Définition : Un **Système d'Exploitation** (SE) est une couche logicielle qui fait l'interface entre le matériel (processeur, mémoire, périphériques) et les applications ou l'utilisateur. Il fournit le bon niveau d'abstraction et gère les ressources de façon équitable et sécurisée.

Remarque : Pour garantir la sécurité et l'équité, les processeurs modernes possèdent au moins deux modes de fonctionnement :

- **Mode Utilisateur (User mode) :** Mode bridé où seules les instructions inoffensives sont exécutées.
- **Mode Noyau (Kernel mode) :** Mode privilégié où toutes les instructions et tous les accès matériels sont autorisés.

La transition entre le mode utilisateur et le mode noyau s'effectue via un **appel système** (*syscall*).

B Espace d'adressage et états d'un processus

Définition : Un **processus** est l'instance dynamique d'un programme en cours d'exécution. Il possède un espace d'adressage propre divisé en :

- **Code (Text) :** Instructions exécutables du programme.
- **Données statiques (Data/BSS) :** Variables globales et statiques.
- **Tas (Heap) :** Zone d'allocation dynamique (*malloc*).
- **Pile (Stack) :** Zone de stockage des variables locales et contextes d'appel.

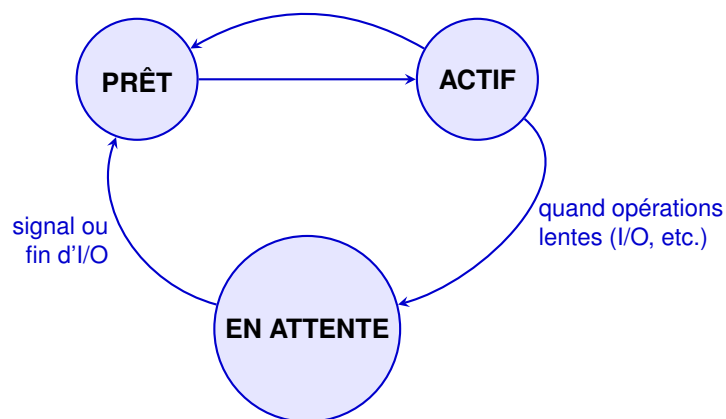


Figure 1: Diagramme des états d'un processus

C Hiérarchie des mémoires

Principe : Hiérarchie de stockage

Les composants de stockage présentent des compromis opposés entre la vitesse d'accès, la capacité et le coût :

1. **Registres** (~ 0.5 ns, très faibles capacités).
2. **Caches** (~ 2 ns).
3. **Mémoire RAM** (~ 10 ns) — volatile.
4. **Stockage secondaire SSD / HDD** (0.1 ms à 10 ms) — non volatile.
5. **Bandes magnétiques** (~ 100 s) — archivage.

II Gestion des Entrées/Sorties et Fichiers

A Descripteurs et tables de fichiers

Définition : Un **descripteur de fichier** (*file descriptor* ou *fd*) est un entier positif renvoyé par un appel système (comme `open`) qui sert d'index à un processus pour accéder à un fichier.

Remarque : Tout processus hérite par défaut de 3 descripteurs ouverts :

- 0 : Entrée standard (`STDIN_FILENO`).
- 1 : Sortie standard (`STDOUT_FILENO`).
- 2 : Sortie d'erreur standard (`STDERR_FILENO`).

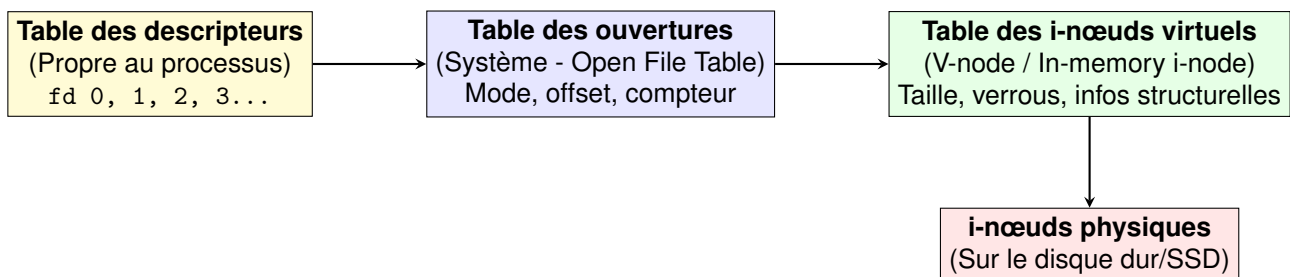


Figure 2: Indirections lors de l'accès à un fichier

B Appels système fondamentaux pour les I/O

Méthode : Manipuler un fichier via les appels système

1. Ouverture (open) :

```
int open(const char *pathname, int flags, mode_t mode);
```

Les flags obligatoires sont `O_RDONLY`, `O_WRONLY`, ou `O_RDWR`. On peut les combiner avec `O_CREAT`, `O_TRUNC`, `O_APPEND`, `O_EXCL` via un OR bit-à-bit (`|`).

2. Lecture (read) :

```
ssize_t read(int fd, void *buf, size_t count);
```

Lit au plus `count` octets et les place dans `buf`. Renvoie le nombre d'octets lus, 0 à la fin du fichier, ou -1 en cas d'erreur.

3. Écriture (write) :

```
ssize_t write(int fd, const void *buf, size_t count);
```

Écrit `count` octets depuis `buf` dans le fichier.

4. Positionnement (lseek) :

```
off_t lseek(int fd, off_t offset, int whence);
```

Modifie la position de la tête de lecture/écriture selon `whence` (`SEEK_SET`, `SEEK_CUR`, `SEEK_END`).

5. Fermeture (close) :

```
int close(int fd);
```

Libère le descripteur et met à jour les compteurs dans les tables système.

C Surcouche FILE* de la bibliothèque standard

Remarque : La bibliothèque standard C (`stdio.h`) propose des fonctions de plus haut niveau manipulant des pointeurs `FILE*`. Un objet `FILE` encapsule un descripteur de fichier et y ajoute un **tampon mémoire (buffer)** pour réduire le nombre d'appels système.

Attention Le buffer d'un flot `FILE*` est vidé (*flushed*) automatiquement dans les cas suivants :

- Quand le tampon est plein.
- Lors d'un appel explicite à `fflush(stream)`.
- À la fermeture avec `fclose(stream)`.
- À la fin normale du processus via `exit` (mais **pas** lors d'un arrêt brutal comme un `SIGKILL` ou un crash).
- Pour les sorties sur terminal, à chaque saut de ligne (`'\n'`).

D Notion d'atomicité

Définition : Une opération est dite **atomique** si elle s'exécute en une seule étape ininterrompue par le système d'exploitation.

Exemple : L'ouverture avec `O_CREAT | O_EXCL` garantit que la vérification de l'existence et la création du fichier se font de façon atomique. De même, l'ouverture en mode `O_APPEND` garantit que le repositionnement en fin de fichier et l'écriture s'effectuent atomiquement.